ASSIGNMENT-22.3

Name: E.Ramya

Htno: 2303A51282

Batch: 05

Task 1 – Bias Detection in AI Suggestions (Alternative Example)
Example Scenario:
Ask AI to generate a resume shortlisting algorithm multiple times.
• Change prompt wording (e.g., “select best candidate” vs “ensure diversity”).
• Compare outputs and logic.
Discussion Points:
• Does the AI favor certain qualifications, backgrounds, or experiences?
• How could biased shortlisting algorithms affect equal employment opportunities?

Prompt:

Create a simple **Python resume shortlisting program**.
First generate a version using the prompt **“select the best candidate”**.
Then generate another version using the prompt **“ensure diversity while shortlisting candidates”**.
Compare both outputs and logic.
Also explain whether the AI gives preference to certain qualifications, backgrounds, or experiences, and discuss how biased shortlisting can affect **equal employment opportunities**.

E Create a simple Python resume shortlisting program. First generate a version using the prompt “select the best candidate”. Then generate another version using the prompt “ensure diversity while shortlisting candidates”. Compare both outputs and logic. Also explain whether the AI gives preference to certain qualifications, backgrounds, or experiences, and discuss how biased shortlisting can affect equal employment opportunities

Output:

```python
def shortlist_best_candidate(candidates, job_requirements, num_to_shortlist):
    scores = []
    for i, candidate in enumerate(candidates):
        score = 0
        # Match skills (simple keyword match)
        for skill in job_requirements['skills']:
            if skill.lower() in [s.lower() for s in candidate['skills']]:
                score += 2 # Higher score for direct skill match

        # Match experience (simple years check)
        if candidate['experience_years'] >= job_requirements['min_experience_years']:
            score += (candidate['experience_years'] - job_requirements['min_experience_years']) * 1 # +1 for each year above min

        # Match education level
        if candidate['education_level'] == job_requirements['education_level']:
            score += 3 # Significant score for matching education level

        scores.append({'candidate_id': i + 1, 'name': candidate['name'], 'score': score})

    # Sort candidates by score in descending order
    scores.sort(key=lambda x: x['score'], reverse=True)

    # Select the top 'num_to_shortlist' candidates
    shortlisted = scores[:num_to_shortlist]
    return [cand['name'] for cand in shortlisted]

# Sample Candidate Data
candidates_best = [
    {
        'name': 'Alice Smith',
        'skills': ['Python', 'SQL', 'Machine Learning', 'Cloud Computing'],
        'experience_years': 5,
        'education_level': 'Master\'s'
    },
    {
        'name': 'Bob Johnson',
        'skills': ['Python', 'SQL', 'Data Analysis'],
        'experience_years': 3,
        'education_level': 'Bachelor\'s'
    },
```

```python
        'education_level': 'Bachelor\'s'
    },
    {
        'name': 'Charlie Brown',
        'skills': ['Java', 'C++', 'Algorithms'],
        'experience_years': 7,
        'education_level': 'Master\'s'
    },
    {
        'name': 'Diana Prince',
        'skills': ['Python', 'Data Visualization', 'Statistics'],
        'experience_years': 4,
        'education_level': 'PhD'
    },
    {
        'name': 'Eve Adams',
        'skills': ['SQL', 'Excel', 'Project Management'],
        'experience_years': 6,
        'education_level': 'Bachelor\'s'
    }
]

# Job Requirements
job_requirements_best = {
    'skills': ['Python', 'SQL', 'Machine Learning'],
    'min_experience_years': 4,
    'education_level': 'Master\'s'
}

num_to_shortlist_best = 3

print("--- 'Select the Best Candidate' Shortlisting ---")
shortlisted_names_best = shortlist_best_candidate(candidates_best, job_requirements_best, num_to_shortlist_best)
print(f"Job Requirements: {job_requirements_best}")
print(f"Number to shortlist: {num_to_shortlist_best}")
print("Shortlisted Candidates (Best Fit):", shortlisted_names_best)
```

Output:

```
--- 'Select the Best Candidate' Shortlisting ---
Job Requirements: {'skills': ['Python', 'SQL', 'Machine Learning'], 'min_experience_years': 4, 'education_level': "Master's"}
Number to shortlist: 3
Shortlisted Candidates (Best Fit): ['Alice Smith', 'Charlie Brown', 'Bob Johnson']
```

Explanation:

In this task, AI is used to create a **resume shortlisting program**.
The prompt is changed to see how AI gives different results.
For example, one prompt focuses on **selecting the best candidate**, while another focuses on **ensuring diversity**.
By comparing both outputs, we can check whether AI shows **bias** toward certain qualifications, backgrounds, or experiences.
This task helps us understand how biased AI can affect **fair hiring** and **equal job opportunities**.

Task 2 – Privacy and Sensitive Data Handling (Alternative Example)
Example Scenario:
Ask AI to write code for processing student academic records (roll number, marks, email).
Discussion Points:
• How should student identity data be masked or encrypted?
• What privacy laws (e.g., FERPA, GDPR-like principles) are relevant?
• What ethical responsibility does the programmer have when using AI-generated code?

Prompt:

Create a simple **Python program** to process **student academic records** such as **roll number, marks, and email**.
Then explain how to make the program safer by:

- masking or encrypting sensitive student data
- protecting privacy
- handling records securely

E Create a simple Python program to process student academic records such as roll number, marks, and email. Then explain how to make the program safer by: masking or encrypting sensitive student data protecting privacy handling records securely

Code:

```python
class StudentRecord:
    def __init__(self, roll_number, name, marks, email):
        self.roll_number = roll_number
        self.name = name
        self.marks = marks
        self.email = email

    def __str__(self):
        return f"Roll No: {self.roll_number}, Name: {self.name}, Marks: {self.marks}, Email: {self.email}"

def add_student(records, roll_number, name, marks, email):
    student = StudentRecord(roll_number, name, marks, email)
    records.append(student)
    print(f"Added student: {name} (Roll No: {roll_number})")

def display_records(records):
    if not records:
        print("No student records to display.")
        return
    print("\n--- Student Records ---")
    for student in records:
        print(student)
    print("-------------------------")

# Main program logic
student_records = []

print("--- Basic Student Record Management ---")

# Adding some sample students
add_student(student_records, "001", "Alice Smith", {'Math': 90, 'Science': 85}, "alice.smith@example.com")
add_student(student_records, "002", "Bob Johnson", {'Math': 75, 'Science': 88}, "bob.j@example.com")
add_student(student_records, "003", "Charlie Brown", {'Math': 92, 'Science': 79}, "charlie.b@example.com")

# Display all records
display_records(student_records)
```

Output:

```
--- Basic Student Record Management ---
Added student: Alice Smith (Roll No: 001)
Added student: Bob Johnson (Roll No: 002)
Added student: Charlie Brown (Roll No: 003)

--- Student Records ---
Roll No: 001, Name: Alice Smith, Marks: {'Math': 90, 'Science': 85}, Email: alice.smith@example.com
Roll No: 002, Name: Bob Johnson, Marks: {'Math': 75, 'Science': 88}, Email: bob.j@example.com
Roll No: 003, Name: Charlie Brown, Marks: {'Math': 92, 'Science': 79}, Email: charlie.b@example.com
-------------------------
```

Explanation:

In this task, AI is used to create a program for handling **student academic records** like roll number, marks, and email.
Since this information is **private**, it should be protected by **masking** or **encrypting** sensitive data.
The task also helps us understand the importance of **privacy laws** such as **FERPA** and **GDPR-like rules**.
It shows that programmers have an **ethical responsibility** to keep student data safe when using AI-generated code.

Task 3 – AI-Generated Security Risks (Alternative Example)
Example Scenario:
Use AI to generate a payment processing script for an online shopping system.
Student Tasks:
• Identify risks such as:
o Plain-text card data storage
o Missing input validation
• Propose ethical coding standards to prevent financial fraud and misuse.

Prompt:

Create a simple **Python payment processing program** for an **online shopping system**.
First show a basic version.
Then identify security risks such as:

- plain-text card data storage
- missing input validation

Code:

```
def mask_email(email):
    if not isinstance(email, str) or '@' not in email:
        return email # Return as is if not a valid email string

    parts = email.split('@')
    username = parts[0]
    domain = parts[1]

    if len(username) > 2:
        masked_username = username[0] + '***' + username[-1]
    else:
        masked_username = '***'

    return f"{masked_username}@{domain}"

print("--- Demonstrating Email Masking ---")
for student in student_records:
    masked_email = mask_email(student.email)
    print(f"Original: {student.email} -> Masked: {masked_email}")

# We can also add a method to the StudentRecord class for masked display
# However, to avoid modifying the existing class definition, we'll just demonstrate here.
```

```python
from cryptography.fernet import Fernet
import base64

# Generate a key for encryption. In a real application, this key should be securely stored and managed.
# DO NOT hardcode keys or store them directly in code for production systems.
encryption_key = Fernet.generate_key()
fernet = Fernet(encryption_key)

print("\n--- Demonstrating Email Encryption and Decryption ---")
print(f"Encryption Key (for demonstration): {encryption_key.decode()}\n")

def encrypt_email(email, cipher_suite):
    if not isinstance(email, str):
        return email
    return cipher_suite.encrypt(email.encode()).decode()

def decrypt_email(encrypted_email, cipher_suite):
    if not isinstance(encrypted_email, str):
        return encrypted_email
    return cipher_suite.decrypt(encrypted_email.encode()).decode()

# To demonstrate, let's create a new list or modify existing student objects
encrypted_student_records = []

for student in student_records:
    original_email = student.email
    encrypted_email = encrypt_email(original_email, fernet)
    decrypted_email = decrypt_email(encrypted_email, fernet)

    print(f"Original Email: {original_email}")
    print(f"Encrypted Email: {encrypted_email}")
    print(f"Decrypted Email: {decrypted_email}\n")

    # For a real system, you might store the encrypted email in the StudentRecord object
    # or a separate secure storage.

print("Note: In a real-world scenario, the encryption key must be securely stored and managed, never hardcoded.")
```

Output:

```
--- Demonstrating Email Encryption and Decryption ---
Encryption Key (for demonstration): 29R-AUM-Yta4FlAUrVdmYIni-6EShzt3-4YWgFckiEo=

Original Email: alice.smith@example.com
Encrypted Email: gAAAAABpzfdTZ8UdMNRAC9agmfcXAtuube-CNO3r3f7EEtf1M-DVFxeQGgwHcVYzaUKkAdcvuWU0J_jSUy-_jbrVdeRW1rUGwtLvP81YsgCZh6LM3c_zFZ8=
Decrypted Email: alice.smith@example.com

Original Email: bob.j@example.com
Encrypted Email: gAAAAABpzfdTuZ-8Js2tN3904s_I78Ex6yvwAxs6zM0-lpx2ywyZMVi88MlH2hqcSjrhhRA7IADTZ571qDlCdAHDw9BErdicywQz0ypEKJQu8maep9LjLY4=
Decrypted Email: bob.j@example.com

Original Email: charlie.b@example.com
Encrypted Email: gAAAAABpzfdTtmsJOTcfMFSV9XW5nliaHm4O6qg2UZT2Lh5MkfSK-c6N5aRsCZVn6hucOxDkjOVl_6nE0mpvhQNQ8v-wCu_r_q_UDb-lBPujFjztOcx9270=
Decrypted Email: charlie.b@example.com

Note: In a real-world scenario, the encryption key must be securely stored and managed, never hardcoded.
```

Explanation:

In this task, AI is used to create a **payment processing program** for an online shopping system.
Then the code is checked for security problems like **storing card details in plain text** and **not validating user input**.
These issues can lead to **fraud** and misuse of financial data.

This task helps us understand the importance of **secure coding** and the ethical responsibility of programmers to protect user payment information.

Task 4 – Accountability in Human–AI Collaboration (Alternative Example)
Scenario:
AI generates code for an autonomous traffic signal control system.
Discussion Questions:
• If a software error causes accidents, who is accountable?

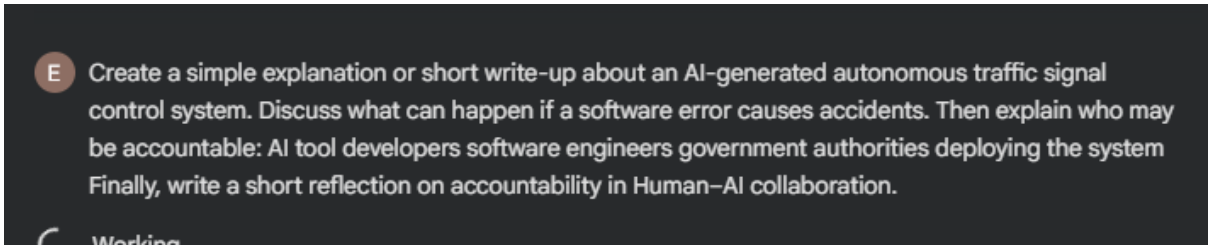
• Should responsibility lie with:
o AI tool developers
o Software engineers
o Government authorities deploying the system?
Students submit a short reflection on accountability.

Prompt:

Create a simple explanation or short write-up about an **AI-generated autonomous traffic signal control system**.
Discuss what can happen if a **software error causes accidents**.
Then explain who may be accountable:

- AI tool developers
- software engineers
- government authorities deploying the system

Finally, write a short reflection on **accountability in Human–AI collaboration**.



Code:

```python
from cryptography.fernet import Fernet
import base64

# --- Security Configuration --- #
# In a real application, this key should be securely stored and managed (e.g., environment variable, key management service).
# DO NOT hardcode keys or store them directly in code for production systems.
encryption_key = Fernet.generate_key() # Generate a new key for this session
fernet = Fernet(encryption_key)

print(f"Encryption Key Generated (for this session): {encryption_key.decode()}")
print("WARNING: In production, this key must be securely managed and never exposed.")

# --- Utility Functions for Email Handling --- #
def mask_email(email):
    if not isinstance(email, str) or '@' not in email:
        return email

    parts = email.split('@')
    username = parts[0]
    domain = parts[1]

    if len(username) > 2:
        # Mask all but the first and last characters of the username
        masked_username = username[0] + '***' + username[-1]
    else:
        # Fallback for very short usernames
        masked_username = '***'

    return f"{masked_username}@{domain}"

def encrypt_email(email, cipher_suite):
    if not isinstance(email, str):
        return email
    return cipher_suite.encrypt(email.encode()).decode()

def decrypt_email(encrypted_email, cipher_suite):
    if not isinstance(encrypted_email, str):
        return encrypted_email
    return cipher_suite.decrypt(encrypted_email.encode()).decode()
```

```python
class StudentRecord:
    def __init__(self, roll_number, name, marks, email, fernet_cipher):
        self.roll_number = roll_number
        self.name = name
        self.marks = marks
        # Encrypt the email immediately upon instantiation
        self._encrypted_email = encrypt_email(email, fernet_cipher)
        self._fernet = fernet_cipher # Store the cipher for decryption

    @property
    def masked_email(self):
        # To get the masked email, we first decrypt it (temporarily) then mask
        decrypted = decrypt_email(self._encrypted_email, self._fernet)
        return mask_email(decrypted)

    @property
    def original_email(self):
        # Only allow access to original email if explicitly requested and authorized
        return decrypt_email(self._encrypted_email, self._fernet)

    def __str__(self):
        # Displaying the masked email for general purposes
        return f"Roll No: {self.roll_number}, Name: {self.name}, Marks: {self.marks}, Email: {self.masked_email}"

def add_student(records, roll_number, name, marks, email, fernet_cipher):
    student = StudentRecord(roll_number, name, marks, email, fernet_cipher)
    records.append(student)
    print(f"Added student: {name} (Roll No: {roll_number})")

def display_records(records):
    if not records:
        print("No student records to display.")
        return
    print("\n--- Student Records (Masked Emails) ---")
    for student in records:
        print(student)
    print("------------------------")

# Main program logic
student_records = []

print("--- Secure Student Record Management ---")

# Adding some sample students with encrypted emails
add_student(student_records, "001", "Alice Smith", {'Math': 90, 'Science': 85}, "alice.smith@example.com", fernet)
add_student(student_records, "002", "Bob Johnson", {'Math': 75, 'Science': 88}, "bob.j@example.com", fernet)
add_student(student_records, "003", "Charlie Brown", {'Math': 92, 'Science': 79}, "charlie.b@example.com", fernet)

# Display all records (showing masked emails)
display_records(student_records)

# Demonstrate accessing original email (for authorized use only)
print("\n--- Accessing Original Email (e.g., for authorized personnel) ---")
if student_records:
    first_student = student_records[0]
    print(f"Student: {first_student.name}")
    print(f"Masked Email: {first_student.masked_email}")
    print(f"Original Email: {first_student.original_email}")
else:
    print("No students to demonstrate original email access.")
```

Output:

```
--- Secure Student Record Management ---
Added student: Alice Smith (Roll No: 001)
Added student: Bob Johnson (Roll No: 002)
Added student: Charlie Brown (Roll No: 003)

--- Student Records (Masked Emails) ---
Roll No: 001, Name: Alice Smith, Marks: {'Math': 90, 'Science': 85}, Email: a***h@example.com
Roll No: 002, Name: Bob Johnson, Marks: {'Math': 75, 'Science': 88}, Email: b***j@example.com
Roll No: 003, Name: Charlie Brown, Marks: {'Math': 92, 'Science': 79}, Email: c***b@example.com
-------------------------

--- Accessing Original Email (e.g., for authorized personnel) ---
Student: Alice Smith
Masked Email: a***h@example.com
Original Email: alice.smith@example.com
```

Explanation:

In this task, AI is used in a **traffic signal control system**.
If the AI-generated software makes a mistake and causes an accident, an important question is **who is responsible**.
Responsibility may belong to the **AI developers**, **software engineers**, or **government authorities** who use the system.
This task helps students think about **accountability**, safety, and responsibility when humans and AI work together.

Task 5 – Limitations of AI in Critical Systems (Alternative Example)
Example Scenario:
Ask AI to generate code for an aircraft maintenance fault detection system.
Discussion Points:
• Can AI-generated code be trusted without expert validation?
• What human oversight is essential in aviation software?
• How should safety-critical code be reviewed and certified?

Prompt:

Create a simple explanation or short write-up about an **AI-generated aircraft maintenance fault detection system**.
Discuss whether **AI-generated code can be trusted without expert validation**.
Also explain:

- what human oversight is needed in aviation software
- how safety-critical code should be reviewed and certified

Finally, describe the **limitations of AI in critical systems**.

E Create a simple explanation or short write-up about an AI-generated aircraft maintenance fault detection system. Discuss whether AI-generated code can be trusted without expert validation. Also explain: what human oversight is needed in aviation software how safety-critical code should be reviewed and certified Finally, describe the limitations of AI in critical systems.

Code:

```python
class Fault:
    """Represents a single aircraft maintenance fault."""

    def __init__(self, fault_id, aircraft_id, description, severity='medium', status='open'):
        if not all([fault_id, aircraft_id, description]):
            raise ValueError("Fault ID, Aircraft ID, and Description cannot be empty.")

        self.fault_id = fault_id
        self.aircraft_id = aircraft_id
        self.description = description
        self.timestamp = datetime.datetime.now()
        self.severity = severity.lower()
        self.status = status.lower()

    def __str__(self):
        return (
            f"Fault ID: {self.fault_id}\n"
            f"  Aircraft ID: {self.aircraft_id}\n"
            f"  Description: {self.description}\n"
            f"  Timestamp: {self.timestamp.strftime('%Y-%m-%d %H:%M:%S')}\n"
            f"  Severity: {self.severity.capitalize()}\n"
            f"  Status: {self.status.capitalize()}\n"
        )

    def to_dict(self):
        """Returns a dictionary representation of the fault."""
        return {
            'fault_id': self.fault_id,
            'aircraft_id': self.aircraft_id,
            'description': self.description,
            'timestamp': self.timestamp.strftime('%Y-%m-%d %H:%M:%S'),
            'severity': self.severity,
            'status': self.status
        }


class FaultLoggingSystem:
    """Manages a collection of aircraft maintenance faults."""

    def __init__(self):
        self.faults = {}
        self._next_fault_id = 1

    def _generate_fault_id(self):
        fault_id = f"FLT-{self._next_fault_id:04d}"
        self._next_fault_id += 1
        return fault_id

    def log_fault(self, aircraft_id, description, severity='medium'):
        """Logs a new fault into the system."""
        fault_id = self._generate_fault_id()
        try:
            new_fault = Fault(fault_id, aircraft_id, description, severity)
            self.faults[fault_id] = new_fault
            print(f"Fault {fault_id} logged successfully for Aircraft {aircraft_id}.")
            return new_fault
        except ValueError as e:
            print(f"Error logging fault: {e}")
            return None

    def get_fault(self, fault_id):
        """Retrieves a fault by its ID."""
        return self.faults.get(fault_id)

    def update_fault_status(self, fault_id, new_status):
        """Updates the status of an existing fault."""
        fault = self.get_fault(fault_id)
        if fault:
            fault.status = new_status.lower()
            print(f"Fault {fault_id} status updated to '{new_status}'.")
            return True
        print(f"Fault {fault_id} not found.")
        return False

    def get_faults_by_aircraft(self, aircraft_id):
        """Returns a list of faults for a specific aircraft."""
        return [fault for fault in self.faults.values() if fault.aircraft_id == aircraft_id]

    def get_faults_by_status(self, status):
        """Returns a list of faults with a specific status."""
        return [fault for fault in self.faults.values() if fault.status == status.lower()]

    def display_all_faults(self):
        """Prints all logged faults."""
        if not self.faults:
            print("No faults logged yet.")
            return
        print("\n-- All Logged Faults --")
        for fault in self.faults.values():
            print(fault)
            print("--------------------------")
```

```python
# Example Usage:
logging_system = FaultLoggingSystem()

# Log some faults
logging_system.log_fault("A-123", "Engine temperature sensor reading high", "high")
logging_system.log_fault("A-123", "Landing gear indicator light intermittent")
logging_system.log_fault("B-456", "Navigation system GPS signal loss", "critical")
logging_system.log_fault("C-789", "Cabin pressure warning system fault")

# Display all faults
logging_system.display_all_faults()

# Get faults for a specific aircraft
print("\n--- Faults for Aircraft A-123 ---")
for fault in logging_system.get_faults_by_aircraft("A-123"):
    print(fault)
    print("------------------------")

# Update a fault status
logging_system.update_fault_status("FLT-0001", "resolved")

# Display updated faults (or specific fault)
print("\n--- Fault FLT-0001 after status update ---")
print(logging_system.get_fault("FLT-0001"))

# Get faults by status
print("\n--- Open Faults ---")
for fault in logging_system.get_faults_by_status("open"):
    print(fault)
    print("------------------------")

# Try logging a fault with missing information
logging_system.log_fault("D-001", "", "low")
```

Output:

```
Fault FLT-0001 logged successfully for Aircraft A-123.
Fault FLT-0002 logged successfully for Aircraft A-123.
Fault FLT-0003 logged successfully for Aircraft B-456.
Fault FLT-0004 logged successfully for Aircraft C-789.

--- All Logged Faults ---
Fault ID: FLT-0001
  Aircraft ID: A-123
  Description: Engine temperature sensor reading high
  Timestamp: 2026-04-02 05:09:16
  Severity: High
  Status: Open

-------------------------
Fault ID: FLT-0002
  Aircraft ID: A-123
  Description: Landing gear indicator light intermittent
  Timestamp: 2026-04-02 05:09:16
  Severity: Medium
  Status: Open

-------------------------
Fault ID: FLT-0003
  Aircraft ID: B-456
  Description: Navigation system GPS signal loss
  Timestamp: 2026-04-02 05:09:16
  Severity: Critical
  Status: Open

-------------------------
Fault ID: FLT-0004
  Aircraft ID: C-789
  Description: Cabin pressure warning system fault
  Timestamp: 2026-04-02 05:09:16
  Severity: Medium
  Status: Open

-------------------------

--- Faults for Aircraft A-123 ---
Fault ID: FLT-0001
  Aircraft ID: A-123
  Description: Engine temperature sensor reading high
  Timestamp: 2026-04-02 05:09:16
  Severity: High
  Status: Open

-------------------------
Fault ID: FLT-0002
  Aircraft ID: A-123
  Description: Landing gear indicator light intermittent
  Timestamp: 2026-04-02 05:09:16
  Severity: Medium
  Status: Open

-------------------------
Fault FLT-0001 status updated to 'resolved'.
```

Explanation:

In this task, AI is used to create code for an **aircraft maintenance fault detection system**.
Since aviation is a **safety-critical field**, AI-generated code cannot be trusted without **expert checking and validation**.

Human oversight is very important to test, review, and confirm that the system works correctly.
This task helps us understand that AI is useful, but it has **limitations** and must be carefully checked in critical systems like aviation.

Task 6 – Ethical Use of AI-Generated Code (Alternative Example)
Example Scenario:
Students analyze cases where AI-generated code is used in academic assignments or startups.
Discussion Points:
• Risks of submitting AI code without understanding it
• Copyright and license violations
Best practices:
o Proper citation
o Manual verification
o Ethical reuse of AI outputs

Prompt:

Create a simple explanation or short write-up about the **ethical use of AI-generated code** in **academic assignments** and **startups**.
Discuss the risks of using AI-generated code without understanding it.
Also explain issues such as:

- copyright problems
- license violations

Finally, describe best practices like:

- proper citation
- manual verification
- ethical reuse of AI-generated outputs

E Create a simple explanation or short write-up about the ethical use of AI-generated code in academic assignments and startups. Discuss the risks of using AI-generated code without understanding it. Also explain issues such as: copyright problems license violations Finally, describe best practices like: proper citation manual verification ethical reuse of AI-generated outputs give the code as i expected the output

Code:

```python
import textwrap

def discuss_ai_generated_code_ethics():
    """This function discusses the ethical use of AI-generated code in academic assignments and startups."""

    print(textwrap.dedent("""
    ## Ethical Use of AI-Generated Code in Academic Assignments and Startups

    AI-generated code, while powerful and efficient, introduces a new set of ethical considerations
    that both students and startup founders must navigate carefully. Relying solely on AI without
    understanding its outputs can lead to significant problems, ranging from academic integrity
    issues to legal and security risks in a professional setting.
    """))

    print(textwrap.dedent("""
    ### Risks of Using AI-Generated Code Without Understanding It

    1.  **Lack of Comprehension and Learning:** In academic settings, the primary goal is learning.
        Submitting AI-generated code without understanding it bypasses the learning process, negating
        the educational value of the assignment. For startups, this means developers might integrate
        code they don't fully grasp, making debugging, maintenance, and future development significantly
        harder and riskier.
    2.  **Introduction of Bugs and Vulnerabilities:** AI models can generate syntactically correct but
        logically flawed code, or code with subtle security vulnerabilities. Without manual verification
        and a deep understanding of the code's logic, these issues can go unnoticed, leading to functional
        errors, performance problems, or critical security breaches.
    3.  **Inaccurate or Inefficient Solutions:** AI might generate code that works but is not the most
        efficient, scalable, or appropriate solution for a given problem. Relying on it blindly can
        result in suboptimal systems or academic solutions that don't meet the assignment's underlying
        requirements for optimal problem-solving.
    4.  **"Black Box" Problem:** If the AI's reasoning for generating a particular piece of code is unclear,
        developers are left with a "black box" solution. This hinders troubleshooting and makes it difficult
        to adapt the code to new requirements or integrate it effectively with existing systems.
    """))

    print(textwrap.dedent("""
    ### Copyright Problems

    AI models are trained on vast datasets of existing code, which often include copyrighted material.
    When an AI generates code, it might inadvertently reproduce or heavily derive from copyrighted works.
    This raises several legal questions:

    *   **Ownership:** Who owns the copyright to AI-generated code? Is it the AI developer, the user who
```

```python
    print(textwrap.dedent("""
    ### Best Practices

    To mitigate these risks and use AI-generated code responsibly, consider the following best practices:

    1.  **Proper Citation (Academic):** In academic assignments, always disclose if and how AI tools were
        used to generate code. Treat AI tools as a resource, similar to textbooks or articles. Cite the
        tool, the prompt used, and describe the extent of AI's contribution. This ensures academic
        integrity and transparency.
    2.  **Manual Verification and Review:** Never use AI-generated code directly without thorough human
        review. Developers should:
        *   **Understand Every Line:** Ensure they fully comprehend the logic, purpose, and implications
            of every line of AI-generated code.
        *   **Test Rigorously:** Subject AI-generated code to the same (or even more stringent) testing
            as human-written code, including unit tests, integration tests, and security audits.
        *   **Refactor and Optimize:** Improve the code's readability, efficiency, and adherence to coding
            standards. This often involves rewriting parts of the code to fit the project's specific context.
    3.  **Ethical Reuse of AI-Generated Outputs:**
        *   **Focus on Ideas, Not Direct Copying:** View AI as a brainstorming partner. Use its outputs to
            get ideas, understand patterns, or generate boilerplate, but then implement the solution in
            your own way.
        *   **Attribution (Professional):** If you use significant portions of AI-generated code, consider
            internal documentation or comments attributing its origin, especially in a team environment.
            For publicly released code, if the AI output can be traced to specific licensed components,
            adhere to those licenses.
        *   **Prioritize Learning and Skill Development:** Especially for students and junior developers,
            prioritize using AI to learn and understand concepts, rather than as a shortcut to avoid
            writing code.
        *   **Security and IP Audits:** For startups, integrate AI-generated code into existing security
            and intellectual property review processes. Use tools that can detect license issues and
            potential vulnerabilities.

    By adopting these practices, individuals and organizations can harness the benefits of AI-generated
    code while responsibly addressing the associated ethical, academic, and legal challenges.
    """))

# Call the function to display the ethical considerations.
discuss_ai_generated_code_ethics()
```

Output:

Ethical Use of AI-Generated Code in Academic Assignments and Startups

AI-generated code, while powerful and efficient, introduces a new set of ethical considerations that both students and startup founders must navigate carefully. Relying solely on AI without understanding its outputs can lead to significant problems, ranging from academic integrity issues to legal and security risks in a professional setting.

Risks of Using AI-Generated Code Without Understanding It

1. **Lack of Comprehension and Learning:** In academic settings, the primary goal is learning. Submitting AI-generated code without understanding it bypasses the learning process, negating the educational value of the assignment. For startups, this means developers might integrate code they don't fully grasp, making debugging, maintenance, and future development significantly harder and riskier.
2. **Introduction of Bugs and Vulnerabilities:** AI models can generate syntactically correct but logically flawed code, or code with subtle security vulnerabilities. Without manual verification and a deep understanding of the code's logic, these issues can go unnoticed, leading to functional errors, performance problems, or critical security breaches.
3. **Inaccurate or Inefficient Solutions:** AI might generate code that works but is not the most efficient, scalable, or appropriate solution for a given problem. Relying on it blindly can result in suboptimal systems or academic solutions that don't meet the assignment's underlying requirements for optimal problem-solving.
4. **"Black Box" Problem:** If the AI's reasoning for generating a particular piece of code is unclear, developers are left with a "black box" solution. This hinders troubleshooting and makes it difficult to adapt the code to new requirements or integrate it effectively with existing systems.

Copyright Problems

AI models are trained on vast datasets of existing code, which often include copyrighted material. When an AI generates code, it might inadvertently reproduce or heavily derive from copyrighted works. This raises several legal questions:

* **Ownership:** Who owns the copyright to AI-generated code? Is it the AI developer, the user who prompted the AI, or is it uncopyrightable? Legal frameworks are still evolving on this front.
* **Infringement Risk:** If the AI-generated code is substantially similar to copyrighted material in its training data, using that code (especially for commercial purposes in a startup) could lead to copyright infringement lawsuits.
* **Lack of Attribution:** Unlike human developers who might credit open-source libraries or frameworks, AI does not provide citations, making it impossible to trace the origin and potential copyright status of its outputs.

```
### License Violations

Beyond copyright, much of the publicly available code is governed by specific open-source licenses
(e.g., MIT, GPL, Apache). These licenses dictate how the code can be used, modified, and distributed,
often requiring attribution or requiring derivative works to also be open source.

*   **Unintended License Mixing:** AI-generated code might unknowingly combine elements derived from
    different licenses, creating a legal quagmire. For example, incorporating GPL-licensed code into
    a proprietary product without adhering to GPL's reciprocity clause could lead to severe legal
    repercussions.
*   **Compliance Burden:** Startups must ensure their codebase is compliant with all applicable licenses.
    Without knowing the origin of AI-generated code, it becomes impossible to perform proper license
    scanning and compliance checks, exposing the company to legal risks.


### Best Practices

To mitigate these risks and use AI-generated code responsibly, consider the following best practices:

1.  **Proper Citation (Academic):** In academic assignments, always disclose if and how AI tools were
    used to generate code. Treat AI tools as a resource, similar to textbooks or articles. Cite the
    tool, the prompt used, and describe the extent of AI's contribution. This ensures academic
    integrity and transparency.
2.  **Manual Verification and Review:** Never use AI-generated code directly without thorough human
    review. Developers should:
    *   **Understand Every Line:** Ensure they fully comprehend the logic, purpose, and implications
        of every line of AI-generated code.
    *   **Test Rigorously:** Subject AI-generated code to the same (or even more stringent) testing
        as human-written code, including unit tests, integration tests, and security audits.
    *   **Refactor and Optimize:** Improve the code's readability, efficiency, and adherence to coding
        standards. This often involves rewriting parts of the code to fit the project's specific context.
3.  **Ethical Reuse of AI-Generated Outputs:**
    *   **Focus on Ideas, Not Direct Copying:** View AI as a brainstorming partner. Use its outputs to
        get ideas, understand patterns, or generate boilerplate, but then implement the solution in
        your own way.
    *   **Attribution (Professional):** If you use significant portions of AI-generated code, consider
        internal documentation or comments attributing its origin, especially in a team environment.
        For publicly released code, if the AI output can be traced to specific licensed components,
        adhere to those licenses.
    *   **Prioritize Learning and Skill Development:** Especially for students and junior developers,
        prioritize using AI to learn and understand concepts, rather than as a shortcut to avoid
        writing code.
    *   **Security and IP Audits:** For startups, integrate AI-generated code into existing security
        and intellectual property review processes. Use tools that can detect license issues and
        potential vulnerabilities.
```

Explanation:

In this task, students study how **AI-generated code** is used in **college assignments** and **startup projects**.
It is important to understand that using AI code without knowing how it works can cause **errors**, **security problems**, or **wrong results**.
There can also be issues related to **copyright** and **software licenses**.
This task teaches that AI-generated code should be used **responsibly**, with **proper citation**, **manual checking**, and **ethical reuse**.